Error Handling & Resilience

Section 12: Building Robust Agents

LangGraph v1.0.x

March 2026

Mental Model: Building Resilient Agents

- Agents run autonomously, often 24/7, in unpredictable environments
- External APIs fail, networks timeout, data is malformed
- Without resilience, a single failure crashes the entire workflow
- Resilient agents gracefully handle failures and recover automatically
- Think *crash recovery*, *retry logic*, *fallback paths*

Key Insight

Resilience is not about preventing failures-it's about detecting them early and recovering fast.

Why Resilience Matters

- **Long-running processes:** Agents may execute for hours or days
- **Unreliable I/O:** Network calls, file operations, database queries can fail
- **Cascading failures:** One bad node can corrupt downstream state
- **Cost:** Retrying expensive operations without limits wastes tokens
- **User experience:** Silent failures or infinite hangs degrade trust

Production Reality

Without error handling, agents will fail in production. Resilience strategies are mandatory, not optional.

Node-Level Error Handling: Try/Except

- Wrap node logic in try/except blocks
- Catch specific exceptions (APIError, TimeoutError)
- Return error state instead of raising
- Log error details for monitoring

```
def search_node(state): try:  results = search_api(...); return {"results":
results} except APIError as e:  return {"error":  str(e)}
```

Retry Policy: Automatic Retries

- LangGraph supports `retry_policy` on node definitions
- Automatically retries a node on transient failures
- Exponential backoff prevents overwhelming failing services
- Configure: `max_attempts`, `backoff_factor`, `retry_on`

When to Use Retries

Ideal for transient failures (network timeouts, rate limits, temporary service outages). Not suitable for logic errors or invalid input.

Configuring RetryPolicy

- Import: `from langgraph.pregel import RetryPolicy`
- Create: `RetryPolicy(max_attempts=3, backoff_factor=2.0)`
- Specify: `retry_on=[APIError, TimeoutError]`
- Attach: `graph.add_node(..., retry_policy=rp)`

Backoff Timing

Exponential backoff: 1s, 2s, 4s, 8s... Prevents overwhelming failing services.

Error Routing: Directing Failures

- Use conditional edges to route on error state
- Direct failed nodes to recovery or error-handling nodes
- Example: if search fails, try fallback data source

Routing logic

```
def route_on_error(state):
  if state.get("error"):
    return "fallback_search"
  return "process_results"
```

Tip

Combine try/except in nodes with error routing in edges for comprehensive error handling.

Fallback Patterns: Graceful Degradation

- Primary approach fails -¿ use secondary (slower, cheaper, less accurate)
- Examples: web search -¿ cached data; API call -¿ local model
- Ensure fallback is always available and tested
- Never degrade silently-log fallback use for monitoring

Fallback Risk

If both primary and fallback fail, the error still propagates. Have a final error handler.

Durable Execution: Checkpoint Recovery

- Checkpointing saves state after each node execution
- If agent crashes, resume from last checkpoint (not from start)
- Dramatically reduces waste and improves reliability
- LangGraph's default Executor uses checkpointing

Durability Magic

A 10-step process that crashes at step 8 resumes at step 8, not step 1. Recovery is instant.

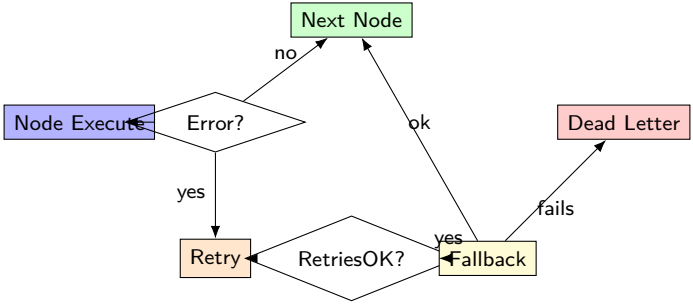
Checkpoint Configuration

- Import: `from langgraph.checkpoint.sqlite import SqliteSaver`
- Create: `saver = SqliteSaver(db_path="agent.db")`
- Compile: `graph = builder.compile(checkpointer=saver)`
- Invoke: `graph.invoke(data, config={"configurable": {"thread_id": ".."}})`

Checkpointer Options

SqliteSaver for single-machine. PostgresSaver for distributed/production deployments.

Error Handling Flow Diagram



[Docs: Error Handling Flow]

Timeout Handling for Long-Running Nodes

- Set node-level timeouts to prevent infinite waits
- LangGraph: `timeout=seconds` on node execution
- Catch `TimeoutError` and route to fallback or error handler

Timeout Strategy

Always set reasonable timeouts on external API calls. Default to fast failure over slow hang.

Timeout Configuration Example

- Set: `graph.add_node("api_call", node_fn, timeout=10)`
- Combine with retry: `retry_policy=RetryPolicy(...)`
- Timeout fires -¿ treated as retriable error
- Max retries enforced; fail gracefully if all attempts exhaust

Timeout Strategy

Set conservative timeouts. Better fast failure than slow hang.

Dead Letter Queues: Capturing Unrecoverable Errors

- When all retries and fallbacks fail, send to "dead letter" queue
- Preserve error state for manual review and debugging
- Example: write to database, file, or monitoring system
- Never silently drop errors

Critical

Dead letter handling is your last defense. Ensure it's robust and monitored.

Dead Letter Pattern

- Capture failed state: {`"timestamp"`, `"agent_id"`, `"error"`, `"state_snapshot"`}
- Persist to database, file, or monitoring system
- Never silently discard errors
- Enable debugging and root cause analysis

Critical

Dead letter is your last safety net. Ensure it's robust and monitored.

Monitoring & Alerting with LangSmith

- LangSmith integrates with LangGraph for error tracking
- Automatic tracing of node execution, inputs, outputs, errors
- Set alerts on error rate thresholds
- Dashboards for latency, token usage, retry frequency

Observability

Instrument error handling with LangSmith. Without visibility, you're flying blind.

LangSmith Tracing

- Set env: `LANGSMITH_API_KEY`, `LANGSMITH_PROJECT`
- All nodes automatically traced (no code changes)
- Logs: errors, latency, token usage, retry counts
- Dashboards show error rates, fallback activations

Zero Friction

Just set env vars; automatic observability kicks in.

Testing Error Scenarios

- Write unit tests that inject failures (mock API errors)
- Test retry logic: verify correct number of retries
- Test fallback activation: ensure fallback is invoked on primary failure
- Test error routing: confirm errors reach error handlers
- Test timeout behavior: verify timeout fires and is caught

Best Practice

Every error path must be tested. Untested error handling is untested code.

Testing Retry Behavior

- Mock API to fail twice, succeed on third attempt
- Verify result contains expected data
- Assert call count = 3 (correct number of retries)
- Test fallback is invoked if all retries exhaust

Test Pattern

Mock external dependencies. Verify retry count, fallback activation.

Common Pitfalls

- **Swallowing errors:** Catching all exceptions, returning safe default, losing context
- **Infinite retries:** No max_attempts; agent retries forever on permanent failures
- **Fallback loops:** Fallback fails too, but no error routing; state corrupted
- **No monitoring:** Errors happen silently; no visibility into failure rates
- **Timeout too long:** Waits 5+ minutes on failed call before failing

[Docs: Error Handling Best Practices]

Summary: Error Handling Strategy

- **Layer 1 (Node-level):** Try/except inside node functions
- **Layer 2 (Automatic):** Retry policies with exponential backoff
- **Layer 3 (Routing):** Conditional edges to fallback nodes
- **Layer 4 (Durability):** Checkpoint recovery from crashes
- **Layer 5 (Capture):** Dead letter handlers for unrecoverable errors
- **Layer 6 (Visibility):** LangSmith monitoring and alerting

Remember

Resilient agents are not built; they are composed from layers of error handling.

Cheat Sheet: Error Handling

Scenario	Solution
Transient API failure	RetryPolicy + backoff
Invalid input	try/except, return error state
Slow external call	Set timeout, route on timeout
Primary source unavailable	Fallback node + conditional edge
Unrecoverable error	Dead letter queue + log
Need visibility	LangSmith tracing + alerts
Crash recovery	Checkpoint + thread_id

Self-Check Questions

1. What's the difference between error routing and retry policies?
2. When would you use a fallback vs. returning an error state?
3. How does checkpointing help with resilience?
4. Why should dead letter errors be logged persistently?
5. How would you test a retry policy with exponential backoff?
6. What's the risk of setting timeouts too long?
7. How does LangSmith visibility improve error handling?